GUÍA TÉCNICA COMPLETA - CAPACITACIÓN REACT NEXT.JS

Arquitectura Web Moderna para Infinite Herbs

Instructor: Jhusseth Sanchez
Fecha de actualización: Febrero/Marzo 2026
Versión: 1.0

Tabla de Contenidos

1. Introducción a la Capacitación
2. Sesión 1: Fundamentos y Arquitectura
3. Sesión 2: Autenticación y Base de Datos
4. Sesión 3: Módulo de Administración de Usuarios
5. Sesión 4: UI/UX Moderna y Despliegue
6. Recursos Adicionales y Referencias
7. Glosario de Términos

1. INTRODUCCIÓN A LA CAPACITACIÓN

1.1 Objetivos Generales

Esta guía técnica proporciona toda la documentación, teoría, enlaces y recursos necesarios para impartir y seguir la capacitación de 20 horas en React Next.js para el equipo de Infinite Herbs.

Propósitos:

- Servir como referencia completa para el instructor durante las sesiones
- Proporcionar material de apoyo teórico y práctico
- Incluir enlaces a documentación oficial y tutoriales de calidad
- Facilitar el seguimiento post-capacitación

1.2 Metodología de Enseñanza

Distribución del tiempo:

- 70% Práctica: Desarrollo en vivo (live coding)
- 20% Teoría: Conceptos y arquitectura
- 10% Q&A: Resolución de dudas

Dinámica de cada sesión:

1. Explicación teórica (15-20 min)
2. Demostración práctica del instructor (60-90 min)
3. Ejercicios hands-on del equipo (30-45 min)
4. Code review y feedback (20-30 min)
5. Sesión de preguntas (15-20 min)

2. SESIÓN 1: FUNDAMENTOS Y ARQUITECTURA

Duración: 4 horas
Día: Miercoles, 8:00 AM - 12:00 PM

2.1 Análisis de Arquitectura Actual (1 hora)

Objetivos

- Entender la arquitectura existente de Infinite Herbs
- Identificar puntos de dolor y oportunidades de mejora
- Establecer la arquitectura objetivo con Next.js

Conceptos Teóricos

Arquitecturas Web Modernas:

Tipo	Ventajas	Desventajas
SPA (Single Page)	Interactividad fluida, menor carga servidor	SEO complicado, bundle inicial grande
SSR (Server-Side Rendering)	SEO excelente, FCP rápido	Mayor carga servidor, complejidad
SSG (Static Generation)	Máxima velocidad, bajo costo hosting	Regeneración requiere rebuild
Híbrida (Next.js)	Combina ventajas de SSR/SSG/CSR	Curva aprendizaje media

Table 1: Comparación de arquitecturas web

¿Por qué Next.js?[1]

Next.js es un framework de React que proporciona:

- Renderizado flexible (SSR, SSG, CSR, ISR)

- File-based routing automático
- Optimización de imágenes built-in
- API routes integradas
- TypeScript support nativo
- Excelente Developer Experience (DX)

Recursos de Aprendizaje

Documentación Oficial:

- Next.js Docs (oficial): https://nextjs.org/docs[2]
- Next.js en Español: https://es.nextjs.im/docs[3]
- Getting Started Guide: https://nextjs.org/docs/app/getting-started

Videos Recomendados:

- "Next.js 15 Tutorial para Principiantes" - Fernando Herrera (1:02h): https://www.youtube.com/watch?v=c_DPfd85qTM[4]
- "Tutorial oficial de NextJS - Capítulo 01" - DevTalles: https://www.youtube.com/watch?v=O6FpxIHOAfI[5]

Ejercicio Práctico

Actividad: Diagrama de arquitectura actual vs objetivo

1. Equipo dibuja arquitectura actual en pizarra/Miro
2. Instructor presenta arquitectura Next.js propuesta
3. Discusión de trade-offs y beneficios
4. Definición de componentes principales del sistema

2.2 Setup del Entorno de Desarrollo (1.5 horas)

Requisitos Previos

Software necesario:

- Node.js 20.x o superior: https://nodejs.org
- Visual Studio Code: https://code.visualstudio.com
- Git: https://git-scm.com

Extensiones VS Code recomendadas:

- ES7+ React/Redux/React-Native snippets
- Prettier - Code formatter
- ESLint
- Tailwind CSS IntelliSense

- GitLens

Creación del Proyecto

Comando de inicialización:

npx create-next-app@latest infinite-herbs-app --typescript --tailwind --app --eslint

Opciones de configuración:

- TypeScript: Sí
- ESLint: Sí
- Tailwind CSS: Sí
- App Router: Sí (nueva arquitectura)
- Import alias: @ (default)

Estructura de Carpetas

Organización recomendada:

```
infinite-herbs-app/
├── app/ # App Router (Next.js 13+)
│   ├── (auth)/ # Grupo de rutas autenticación
│   │   ├── login/
│   │   └── register/
│   ├── (dashboard)/ # Grupo de rutas dashboard
│   │   ├── users/
│   │   └── settings/
│   ├── api/ # API Routes
│   │   ├── auth/
│   │   └── users/
│   ├── layout.tsx # Layout raíz
│   └── page.tsx # Página home
├── components/ # Componentes reutilizables
│   ├── ui/ # Componentes UI (shadcn)
│   ├── forms/
│   └── layouts/
├── lib/ # Utilidades y configuración
│   ├── auth.ts
│   ├── db.ts
│   └── utils.ts
├── types/ # Tipos TypeScript
├── public/ # Assets estáticos
├── .env.local # Variables de entorno
├── next.config.js # Configuración Next.js
├── tailwind.config.ts # Configuración Tailwind
└── tsconfig.json # Configuración TypeScript
```

Configuración de Herramientas

ESLint (.eslintrc.json):

```
{
  "extends": ["next/core-web-vitals", "prettier"],
  "rules": {
    "react/no-unescaped-entities": "off",
    "@next/next/no-page-custom-font": "off"
  }
}
```

Prettier (.prettierrc):

```
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 80,
  "tabWidth": 2
}
```

Recursos de Aprendizaje

Documentación:

- Project Structure: https://nextjs.org/docs/getting-started/project-structure
- Configuration: https://nextjs.org/docs/app/api-reference/next-config-js

Videos:

- "Instalación del Framework Next.js" (timestamp 01:17-06:45 del video anterior)[4]

Ejercicio Práctico

Actividad: Setup completo en máquinas locales

1. Cada participante crea proyecto Next.js
2. Configuran Git y primer commit
3. Instalan extensiones VS Code
4. Verifican que servidor dev funciona (npm run dev)
5. Exploran estructura de carpetas creada

2.3 Fundamentos de React y Next.js (2 horas)

Conceptos Fundamentales de React

Componentes:

React se basa en componentes reutilizables. Existen dos tipos principales:

Functional Components (modernos):

```
// Componente simple
export default function Welcome({ name }: { name: string }) {
```

```
  return Hola, {name};
}
```

Props y State:

- Props: Datos inmutables pasados de padre a hijo
- State: Datos mutables internos del componente

Server Components vs Client Components

Concepto revolucionario de Next.js 13+[6][7]

Aspecto	Server Components	Client Components
Renderizado	Solo en servidor	Servidor (SSR) + Cliente
JavaScript	No se envía al cliente	Se envía al navegador
Hooks React	No permitidos	useState, useEffect, etc.
Eventos	No onClick, onChange	Sí, eventos completos
APIs Browser	No disponibles	window, document, etc.
Acceso DB	Directo	Solo vía API

Table 2: Server vs Client Components

¿Cuándo usar cada uno?

Server Components (por defecto):

- Fetching de datos desde DB/APIs
- Lógica backend pesada
- Contenido estático o con datos del servidor
- Reducir bundle JavaScript del cliente

Client Components (con "use client"):

- Interactividad (clicks, formularios, animaciones)
- Hooks de React (useState, useEffect, useContext)
- APIs del navegador (localStorage, geolocation)
- Librerías que dependen del DOM

Ejemplo práctico:

```
// app/users/page.tsx - Server Component (default)
async function getUsers() {
  const res = await fetch('https://api.example.com/users');
  return res.json();
}
```

```
export default async function UsersPage() {
  const users = await getUsers(); // Fetch directo en servidor

  return (Lista de Usuarios{/* Client Component */});
}

// components/UserList.tsx - Client Component
'use client'; // Directiva obligatoria

import { useState } from 'react';

export function UserList({ users }) {
const [filter, setFilter] = useState('');

const filtered = users.filter(u =>
u.name.toLowerCase().includes(filter.toLowerCase())
);return (<inputvalue={filter}
onChange={(e) => setFilter(e.target.value)}placeholder="Buscar
usuario..."/>{filtered.map(user => ({user.name}))});
}
```

App Router vs Pages Router

Next.js tiene dos sistemas de routing:

Característica	Pages Router (legacy)	App Router (moderno)
Carpeta	pages/	app/
Archivos ruta	pagina.tsx	page.tsx
Layouts	_app.tsx manual	layout.tsx automático
Server Components	No nativo	Sí, por defecto
Data fetching	getServerSideProps	async/await directo
Streaming	Limitado	Completo con Suspense

Table 3: Pages Router vs App Router

Routing dinámico en App Router:

```
app/
├── page.tsx # /
├── about/
│   └── page.tsx # /about
├── blog/
│   ├── page.tsx # /blog
│   └── [slug]/
│   └── page.tsx # /blog/mi-articulo
└── dashboard/
├── layout.tsx # Layout compartido
├── page.tsx # /dashboard
└── users/
```

```
├── page.tsx # /dashboard/users
└── [id]/
└── page.tsx # /dashboard/users/123
```

Layouts Anidados:

```
// app/dashboard/layout.tsx
export default function DashboardLayout({children,}: {
  children: React.ReactNode;
}) {
  return ({/* Sidebar /}{children} {/ Contenido de páginas hijas */}
);
}
```

Data Fetching Patterns

Tres estrategias principales:

1. Server-Side Rendering (SSR) - Por defecto:

```
// Cada request genera HTML fresco
async function getData() {
  const res = await fetch('https://api.example.com/data', {
    cache: 'no-store' // Forzar fetch en cada request
  });
  return res.json();
}

export default async function Page() {
  const data = await getData();
  return{data.title};
}
```

2. Static Site Generation (SSG):

```
// Pre-renderizado en build time
async function getData() {
  const res = await fetch('https://api.example.com/data', {
    next: { revalidate: 3600 } // Revalidar cada hora (ISR)
  });
  return res.json();
}

export default async function Page() {
  const data = await getData();
  return{data.title};
}
```

3. Client-Side Fetching:

```
'use client';

import { useEffect, useState } from 'react';
```

```
export default function Page() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('/api/data')
    .then(res => res.json())
    .then(setData);
  }, []);

  if (!data) return Loading...;
  return{data.title};
}
```

Recursos de Aprendizaje

Documentación Oficial:

- Server Components: https://react.dev/reference/rsc/server-components[8]
- App Router: https://nextjs.org/docs/app
- Data Fetching: https://nextjs.org/docs/app/building-your-application/data-fetching
- Routing: https://nextjs.org/docs/app/building-your-application/routing

Artículos:

- "React Server Components: A Practical Guide for 2026": https://inhaq.com/blog/react-server-components-practical-guide-2026[9]
- "React Server Components con Next.js 16": https://nexgen.mx/blog/react-server-components-nextjs[10]

Videos:

- "Server y Client Components" (timestamp 41:44-52:50 del tutorial Fernando Herrera)[4]
- "Fetch desde Server Components" (timestamp 01:02:28)[4]

Ejercicio Práctico

Actividad 1: Crear componentes Server y Client

1. Crear página que fetch datos de API pública (jsonplaceholder)
2. Implementar filtro interactivo (Client Component)
3. Mostrar datos en tabla (Server Component)
4. Experimentar moviendo lógica entre server/client

Actividad 2: Routing dinámico

1. Crear rutas /products y /products/[id]
2. Implementar layout compartido con navegación
3. Probar navegación entre páginas

2.4 Variables de Entorno y Configuración (0.5 horas)

Gestión de Variables de Entorno

Archivos de configuración:

.env.local # Desarrollo local (no commitear)
.env.development # Desarrollo
.env.production # Producción
.env.test # Testing

Ejemplo .env.local:

```
# Database

DATABASE_URL="Server=localhost;Database=InfiniteHerbs;Trusted_Connec
tion=True;"

API Keys (NUNCA exponer al cliente)

API_SECRET_KEY="tu-secret-key-aqui"

#Variables públicas (expuestas al cliente con NEXT_PUBLIC_)

NEXT_PUBLIC_API_URL="https://api.infiniteherbs.com"
NEXT_PUBLIC_SITE_NAME="Infinite Herbs"
```

Uso en código:

// Server Component - Acceso a todas las variables

```
export default async function ServerPage() {
  const dbUrl = process.env.DATABASE_URL; // ✅ Seguro, solo
  const apiKey = process.env.API_SECRET_KEY; // ✅ Seguro

  return Server data;
}
```

// Client Component - Solo NEXT_PUBLIC_*

```
'use client';

export default function ClientPage() {
  const apiUrl = process.env.NEXT_PUBLIC_API_URL; // ✅ OK
  // const secret = process.env.API_SECRET_KEY; // ❌ undefined

  return API URL: {apiUrl};
}
```

Configuración de next.config.js

Opciones importantes:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  // Dominios permitidos para imágenes
  images: {
    domains: ['api.infiniteherbs.com', 'cdn.example.com'],
  },

  // Redirecciones
  async redirects() {
    return [
      {
        source: '/old-path',
        destination: '/new-path',
        permanent: true,
      },
    ];
  },

  // Headers personalizados (seguridad)
  async headers() {
    return [
      {
        source: '/:path*',
        headers: [
          {
            key: 'X-Frame-Options',
            value: 'DENY',
          },
          {
            key: 'X-Content-Type-Options',
            value: 'nosniff',
          },
        ],
      },
    ];
  },

  // Variables de entorno expuestas
  env: {
    CUSTOM_KEY: process.env.CUSTOM_KEY,
  },
};

module.exports = nextConfig;
```

Buenas Prácticas de Seguridad

- ❌ NUNCA commitear .env.local a Git
- ✅ Usar .env.example como plantilla (sin valores reales)
- ❌ NUNCA exponer API keys con NEXT_PUBLIC_
- ✅ Rotar secrets regularmente

- ✅ Usar variables diferentes por entorno
- ✅ Validar variables requeridas al inicio de app

Validación de variables (lib/env.ts):

```
const requiredEnvVars = [
  'DATABASE_URL',
  'API_SECRET_KEY',
  'NEXT_PUBLIC_API_URL',
] as const;

requiredEnvVars.forEach((envVar) => {
  if (!process.env[envVar]) {
    throw new Error(Missing required environment variable:
${envVar});
  }
});
```

Recursos de Aprendizaje

Documentación:

- Environment Variables: https://nextjs.org/docs/app/building-your-application/configuring/environment-variables
- next.config.js: https://nextjs.org/docs/app/api-reference/next-config-js

Ejercicio Práctico

Actividad: Configurar variables de entorno

1. Crear archivo .env.local con variables de prueba
2. Configurar next.config.js con imagen externa
3. Crear componente que use variable NEXT_PUBLIC_*
4. Verificar que variables privadas no se exponen al cliente

---

3. SESIÓN 2: AUTENTICACIÓN Y BASE DE DATOS

Duración: 4 horas
Día: Jueves, 8:00 AM - 12:00 PM

3.1 Integración con Backend (Entity Framework) (1.5 horas)

Conceptos de Arquitectura Backend

Entity Framework Core es un ORM (Object-Relational Mapper) para .NET que permite trabajar con bases de datos usando objetos C# en lugar de SQL directo[11].

Ventajas:

- Abstracción de base de datos
- Migraciones automáticas
- Type safety con C#
- LINQ queries (consultas tipo-seguras)
- Soporte múltiples DB (SQL Server, PostgreSQL, MySQL)

Configuración de Backend .NET

Instalación de paquetes NuGet:

dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools

DbContext (backend):

```
// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
public AppDbContext(DbContextOptions<AppDbContext> options)
: base(options) { }

public DbSet<User> Users { get; set; }
public DbSet<Role> Roles { get; set; }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Configuración de relaciones
    modelBuilder.Entity<User>()
        .HasOne(u => u.Role)
        .WithMany(r => r.Users)
        .HasForeignKey(u => u.RoleId);
}

}
```

Connection String (appsettings.json):

```
{
"ConnectionStrings": {
  "DefaultConnection":
"Server=localhost;Database=InfiniteHerbs;Trusted_Connection=True;Tru
stServerCertificate=True;"
```

```
}
}
```

Configuración en Program.cs:

```
builder.Services.AddDbContext<AppDbContext>(options =>
  options.UseSqlServer(
    builder.Configuration.GetConnectionString("DefaultConnection")
  )
);
```

API Routes en Next.js (Route Handlers)

Next.js permite crear APIs dentro del proyecto:

```
// app/api/users/route.ts
import { NextRequest, NextResponse } from 'next/server';

// GET /api/users
export async function GET(request: NextRequest) {
try {
// Fetch desde backend .NET
  const response = await fetch('http://localhost:5000/api/users', {
  headers: {
    'Authorization': Bearer ${token},
  },
});

const data = await response.json();
return NextResponse.json(data);


} catch (error) {
    return NextResponse.json(
      { error: 'Error fetching users' },
      { status: 500 }
    );
  }
}

// POST /api/users
export async function POST(request: NextRequest) {
const body = await request.json();

const response = await fetch('http://localhost:5000/api/users', {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'Authorization': Bearer ${token},
},
body: JSON.stringify(body),
});
```

```
const data = await response.json();
return NextResponse.json(data);
}
```

Configuración de CORS

En backend .NET (Program.cs):

```
builder.Services.AddCors(options =>
{
options.AddPolicy("NextJsApp", policy =>
{
policy.WithOrigins("http://localhost:3000") // Next.js dev
.AllowAnyHeader()
.AllowAnyMethod()
.AllowCredentials();
});
});

// Después de app builder
app.UseCors("NextJsApp");
```

Recursos de Aprendizaje

Documentación:

- Entity Framework Core Docs: https://learn.microsoft.com/es-es/ef/core/
- Tutorial EF Core + SQL Server: https://learn.microsoft.com/es-es/aspnet/core/data/ef-mvc/intro[12]
- Next.js Route Handlers: https://nextjs.org/docs/app/building-your-application/routing/route-handlers

Videos:

- "Using SQL Server with Entity Framework Core": https://www.youtube.com/watch?v=xMktEpPmadI[13]
- "Entity Framework Core with .NET 10": https://www.youtube.com/watch?v=RXXrp7eBIhs[14]

Artículos:

- ".NET 5.0 - Connect to SQL Server with EF Core": https://jasonwatmore.com/post/2021/10/21/net-5-connect-to-sql-server-with-entity-framework-core[15]

Ejercicio Práctico

Actividad: Setup backend + API routes

1. Configurar conexión a SQL Server en backend .NET
2. Crear modelo User con Entity Framework
3. Ejecutar migración: dotnet ef migrations add Initial

4. Crear API route en Next.js que consuma endpoint .NET
5. Probar comunicación frontend-backend

3.2 Sistema de Autenticación (2 horas)

Conceptos de Autenticación Web

Diferencia entre Autenticación y Autorización:

Aspecto	Autenticación	Autorización
Pregunta	¿Quién eres?	¿Qué puedes hacer?
Proceso	Login con credenciales	Verificación de permisos
Resultado	Token/Session	Acceso permitido/denegado
Ejemplo	Usuario/contraseña válidos	Rol admin puede borrar

Table 4: Autenticación vs Autorización

JSON Web Tokens (JWT):

Un JWT consta de tres partes separadas por puntos:

header.payload.signature

Ejemplo decodificado:

```
// Header
{
"alg": "HS256",
"typ": "JWT"
}

// Payload
{
"sub": "1234567890",
"name": "John Doe",
"email": "john@example.com",
"role": "admin",
"iat": 1516239022,
"exp": 1516242622
}

// Signature (firmado con secret)
HMACSHA256(
base64UrlEncode(header) + "." + base64UrlEncode(payload),
secret
)
```

Implementación con NextAuth.js

NextAuth.js es la solución oficial para autenticación en Next.js[16][17].

Instalación:

npm install next-auth

Configuración (app/api/auth/[...nextauth]/route.ts):

```
import NextAuth from 'next-auth';
import CredentialsProvider from 'next-auth/providers/credentials';
import { compare } from 'bcrypt';

const handler = NextAuth({
providers: [
CredentialsProvider({
name: 'Credentials',
credentials: {
email: { label: "Email", type: "email" },
password: { label: "Password", type: "password" }
},
async authorize(credentials) {
if (!credentials?.email || !credentials?.password) {
return null;
}

    // Fetch user desde backend/DB
    const response = await
fetch('http://localhost:5000/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        email: credentials.email,
        password: credentials.password,
      }),
    });

    const user = await response.json();

    if (response.ok && user) {
      return {
        id: user.id,
        email: user.email,
        name: user.name,
        role: user.role,
      };
    }

    return null;
  },
}),


],
```

```
callbacks: {
async jwt({ token, user }) {
if (user) {
token.role = user.role;
token.id = user.id;
}
return token;
},

async session({ session, token }) {
  if (session?.user) {
    session.user.id = token.id;
    session.user.role = token.role;
  }
  return session;
},


},

pages: {
signIn: '/login',
},

session: {
strategy: 'jwt',
maxAge: 30 * 24 * 60 * 60, // 30 días
},

secret: process.env.NEXTAUTH_SECRET,
});

export { handler as GET, handler as POST };
```

Formulario de Login

Validación con React Hook Form + Zod:

npm install react-hook-form zod @hookform/resolvers

Schema de validación (lib/schemas.ts):

```
import { z } from 'zod';

export const loginSchema = z.object({
email: z.string()
.email('Email inválido')
.min(1, 'Email requerido'),
password: z.string()
.min(6, 'Contraseña debe tener mínimo 6 caracteres')
.max(100, 'Contraseña muy larga'),
});

export type LoginFormData = z.infer<typeof loginSchema>;
```

Componente Login (app/login/page.tsx):

```
'use client';

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { signIn } from 'next-auth/react';
import { useRouter } from 'next/navigation';
import { loginSchema, LoginFormData } from '@/lib/schemas';

export default function LoginPage() {
const router = useRouter();
const {
register,
handleSubmit,
formState: { errors, isSubmitting },
} = useForm<LoginFormData>({
resolver: zodResolver(loginSchema),
});

const onSubmit = async (data: LoginFormData) => {
const result = await signIn('credentials', {
redirect: false,
email: data.email,
password: data.password,
});

if (result?.errorHola Juan como) {
  alert('Credenciales inválidas');
} else {
  router.push('/dashboard');
}


};

return (
<div className="flex min-h-screen items-center justify-center">
<form onSubmit={handleSubmit(onSubmit)} className="w-full max-w-md
space-y-4 p-8 bg-white rounded-lg shadow" >
```

Iniciar Sesión

```
    <div>
      <label className="block text-sm font-medium mb-1">
        Email
      </label>
      <input
        {...register('email')}
        type="email"
        className="w-full px-3 py-2 border rounded"
      />
      {errors.email && (
        <p className="text-red-500 text-sm mt-1">
```

```
          {errors.email.message}
        </p>
      )}
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">
        Contraseña
      </label>
      <input
        {...register('password')}
        type="password"
        className="w-full px-3 py-2 border rounded"
      />
      {errors.password && (
        <p className="text-red-500 text-sm mt-1">
          {errors.password.message}
        </p>
      )}
    </div>

    <button
      type="submit"
      disabled={isSubmitting}
      className="w-full bg-blue-600 text-white py-2 rounded
hover:bg-blue-700 disabled:opacity-50"
    >
      {isSubmitting ? 'Iniciando...' : 'Iniciar Sesión'}
    </button>
  </form>
</div>


);
}
```

Protección de Rutas con Middleware

Middleware global (middleware.ts en raíz):

```
import { withAuth } from 'next-auth/middleware';
export default withAuth({
callbacks: {
authorized: ({ req, token }) => {
// Verificar si hay token (usuario autenticado)
if (!token) return false;
  // Rutas admin requieren role "admin"
  if (req.nextUrl.pathname.startsWith('/admin')) {
    return token.role === 'admin';
  }

  return true;
},
```

```
},
pages: {
signIn: '/login',
},
});
export const config = {
matcher: ['/dashboard/:path*', '/admin/:path*', '/profile/:path*'],
};
```

Refresh Tokens

Para sesiones de larga duración:

```
// lib/auth.ts
export async function refreshAccessToken(token: JWT) {
try {
const response = await
fetch('http://localhost:5000/api/auth/refresh', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ refreshToken: token.refreshToken }),
});
const refreshedTokens = await response.json();

if (!response.ok) throw refreshedTokens;

return {
  ...token,
  accessToken: refreshedTokens.accessToken,
  accessTokenExpires: Date.now() + refreshedTokens.expiresIn * 1000,
  refreshToken: refreshedTokens.refreshToken ?? token.refreshToken,
};

} catch (error) {
return {
...token,
error: 'RefreshAccessTokenError',
};
}
}
```

Recursos de Aprendizaje

Documentación:

- NextAuth.js Docs: https://next-auth.js.org
- React Hook Form: https://react-hook-form.com
- Zod Validation: https://zod.dev

Artículos:

- "Autenticación Segura en Next.js con NextAuth.js y JWT": https://nexgen.mx/blog/nextjs-authentication-nextauth[16]
- "Autenticación con NextAuth.js": https://www.nexgen.mx/blog/nextjs-authentication-nextauth[17]

Videos:

- "Tutorial Next Auth V5 - Credentials, JWT, Roles": https://www.youtube.com/watch?v=vkV4wApku5s[18]

Ejercicio Práctico

Actividad: Implementar sistema completo de login

1. Instalar NextAuth.js y dependencias
2. Configurar provider de credenciales
3. Crear formulario de login con validación
4. Implementar middleware para proteger rutas
5. Probar login exitoso y fallido
6. Verificar protección de ruta /dashboard

3.3 Gestión de Estado Global (1 hora)

Opciones de State Management

Librería	Ventajas	Cuándo usar
Context API	Nativo React, sin deps	Estado simple/medio
Zustand	Minimal, fácil setup	Estado medio, no requiere Redux
Redux Toolkit	Robusto, DevTools	Apps grandes, estado complejo

Table 5: Comparación de state management

Implementación con Zustand (Recomendado)

Instalación:

npm install zustand

Store de autenticación (lib/store/authStore.ts):

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
interface User {
id: string;
email: string;
name: string;
role: string;
}
interface AuthState {
user: User | null;
token: string | null;
isAuthenticated: boolean;
login: (user: User, token: string) => void;
logout: () => void;
updateUser: (user: Partial<User>) => void;
}
export const useAuthStore = create<AuthState>()(
persist(
(set) => ({
user: null,
token: null,
isAuthenticated: false,
login: (user, token) =>
    set({ user, token, isAuthenticated: true }),
```

```
  logout: () =>
    set({ user: null, token: null, isAuthenticated: false }),

  updateUser: (userData) =>
    set((state) => ({
      user: state.user ? { ...state.user, ...userData } : null,
    })),
}),
{
  name: 'auth-storage', // LocalStorage key
}
)
);
```

Uso en componentes:

```
'use client';
import { useAuthStore } from '@/lib/store/authStore';
export default function UserProfile() {
const { user, logout } = useAuthStore();
if (!user) return
No autenticado
;
return (

Bienvenido, {user.name}
Email: {user.email}
Rol: {user.role}
Cerrar Sesión

);
}
```

Recursos de Aprendizaje

Documentación:

- Zustand: https://docs.pmnd.rs/zustand/getting-started/introduction
- Context API: https://react.dev/learn/passing-data-deeply-with-context

Ejercicio Práctico

Actividad: Implementar store global

1. Instalar Zustand
2. Crear authStore con login/logout
3. Integrar con formulario de login
4. Crear componente que muestre usuario logueado
5. Probar persistencia (recargar página)

3.4 Manejo de Errores y Feedback (0.5 horas)

Error Boundaries

Error Boundary (components/ErrorBoundary.tsx):

```
'use client';
import { Component, ReactNode } from 'react';
interface Props {
children: ReactNode;
fallback?: ReactNode;
}
interface State {
hasError: boolean;
error?: Error;
}
export class ErrorBoundary extends Component<Props, State> {
constructor(props: Props) {
super(props);
this.state = { hasError: false };
}
static getDerivedStateFromError(error: Error): State {
return { hasError: true, error };
}
componentDidCatch(error: Error, errorInfo: any) {
console.error('Error caught by boundary:', error, errorInfo);
}
render() {
if (this.state.hasError) {
return (
this.props.fallback || (

Algo salió mal
{this.state.error?.message}

)
);
}
return this.props.children;

}
}
```

Notificaciones Toast

Con react-hot-toast:

npm install react-hot-toast

Setup (app/layout.tsx):

```
import { Toaster } from 'react-hot-toast';
```

```
export default function RootLayout({ children }) {
return (

{children}

);
}
```

Uso:

```
import toast from 'react-hot-toast';
// Success
toast.success('Usuario creado correctamente');
// Error
toast.error('Error al guardar datos');
// Loading
const loadingToast = toast.loading('Guardando...');
// Después:
toast.dismiss(loadingToast);
toast.success('Guardado!');
// Custom
toast.custom((t) => (
Contenido personalizado
));
```

Recursos de Aprendizaje

Documentación:

- Error Boundaries: https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary
- react-hot-toast: https://react-hot-toast.com

Ejercicio Práctico

Actividad: Agregar manejo de errores

1. Instalar react-hot-toast
2. Agregar Toaster en layout
3. Mostrar toast en login exitoso/fallido
4. Crear ErrorBoundary
5. Probar con componente que lanza error

4. SESIÓN 3: MÓDULO DE ADMINISTRACIÓN DE USUARIOS

Duración: 5 horas
Día: Jueves, 8:00 AM - 1:00 PM

4.1 Diseño de la Base de Datos (0.5 horas)

Modelado de Tablas

Entidades principales:

```
-- Tabla Users
CREATE TABLE Users (
Id INT PRIMARY KEY IDENTITY(1,1),
Email NVARCHAR(255) NOT NULL UNIQUE,
PasswordHash NVARCHAR(MAX) NOT NULL,
FirstName NVARCHAR(100) NOT NULL,
LastName NVARCHAR(100) NOT NULL,
RoleId INT NOT NULL,
IsActive BIT DEFAULT 1,
CreatedAt DATETIME2 DEFAULT GETDATE(),
UpdatedAt DATETIME2 DEFAULT GETDATE(),
CONSTRAINT FK_Users_Roles FOREIGN KEY (RoleId) REFERENCES Roles(Id)
);
-- Tabla Roles
CREATE TABLE Roles (
Id INT PRIMARY KEY IDENTITY(1,1),
Name NVARCHAR(50) NOT NULL UNIQUE,
Description NVARCHAR(255),
CreatedAt DATETIME2 DEFAULT GETDATE()
);
-- Tabla Permissions
CREATE TABLE Permissions (
Id INT PRIMARY KEY IDENTITY(1,1),
Name NVARCHAR(100) NOT NULL UNIQUE,
Resource NVARCHAR(50) NOT NULL,
Action NVARCHAR(50) NOT NULL
);
-- Tabla RolePermissions (many-to-many)
CREATE TABLE RolePermissions (
RoleId INT NOT NULL,
PermissionId INT NOT NULL,
PRIMARY KEY (RoleId, PermissionId),
CONSTRAINT FK_RolePermissions_Roles FOREIGN KEY (RoleId) REFERENCES
Roles(Id),
CONSTRAINT FK_RolePermissions_Permissions FOREIGN KEY (PermissionId)
REFERENCES Permissions(Id)
);
Migraciones con Entity Framework
Crear migración:
dotnet ef migrations add CreateUsersTables
Aplicar migración:
dotnet ef database update
Seed data inicial (DbContext):
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
// Seed roles
```

```
modelBuilder.Entity<Role>().HasData(
new Role { Id = 1, Name = "Admin", Description = "Administrator" },
new Role { Id = 2, Name = "User", Description = "Regular user" }
);
// Seed permissions
modelBuilder.Entity<Permission>().HasData(
    new Permission { Id = 1, Name = "users.create", Resource =
"users", Action = "create" },
    new Permission { Id = 2, Name = "users.read", Resource =
"users", Action = "read" },
    new Permission { Id = 3, Name = "users.update", Resource =
"users", Action = "update" },
    new Permission { Id = 4, Name = "users.delete", Resource =
"users", Action = "delete" }
);

}
```

Ejercicio Práctico

Actividad: Crear esquema de base de datos

Crear modelos en Entity Framework

Generar y aplicar migración

Verificar tablas en SQL Server Management Studio

Insertar datos de prueba manualmente

4.2 API REST para Usuarios (1.5 horas)

Endpoints CRUD Completos

Backend .NET Controller (Controllers/UsersController.cs):

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
private readonly AppDbContext _context;
public UsersController(AppDbContext context)
{
    _context = context;
}

// GET: api/users?page=1&pageSize=10&search=john
[HttpGet]
public async Task<ActionResult<PagedResult<UserDto>>> GetUsers(
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 10,
    [FromQuery] string? search = null
)
{
    var query = _context.Users
        .Include(u => u.Role)
        .Where(u => u.IsActive);

    if (!string.IsNullOrEmpty(search))
    {
        query = query.Where(u =>
            u.FirstName.Contains(search) ||
```

```
            u.LastName.Contains(search) ||
            u.Email.Contains(search)
        );
    }

    var total = await query.CountAsync();
    var users = await query
        .OrderByDescending(u => u.CreatedAt)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(u => new UserDto
        {
            Id = u.Id,
            Email = u.Email,
            FirstName = u.FirstName,
            LastName = u.LastName,
            RoleName = u.Role.Name,
            IsActive = u.IsActive,
            CreatedAt = u.CreatedAt
        })
        .ToListAsync();

    return Ok(new PagedResult<UserDto>
    {
        Items = users,
        TotalCount = total,
        Page = page,
        PageSize = pageSize
    });
}

// GET: api/users/5
[HttpGet("{id}")]
public async Task<ActionResult<UserDto>> GetUser(int id)
{
    var user = await _context.Users
        .Include(u => u.Role)
        .FirstOrDefaultAsync(u => u.Id == id);

    if (user == null)
        return NotFound();

    return Ok(new UserDto
    {
        Id = user.Id,
        Email = user.Email,
        FirstName = user.FirstName,
        LastName = user.LastName,
        RoleName = user.Role.Name,
        IsActive = user.IsActive
    });
}

// POST: api/users
```

```
[HttpPost]
public async Task<ActionResult<UserDto>> CreateUser(CreateUserDto
dto)
{
    // Validar email único
    if (await _context.Users.AnyAsync(u => u.Email == dto.Email))
    {
        return BadRequest("Email already exists");
    }

    // Hash password
    var passwordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password);

    var user = new User
    {
        Email = dto.Email,
        PasswordHash = passwordHash,
        FirstName = dto.FirstName,
        LastName = dto.LastName,
        RoleId = dto.RoleId,
        IsActive = true,
        CreatedAt = DateTime.UtcNow
    };

    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(GetUser), new { id = user.Id },
user);
}

// PUT: api/users/5
[HttpPut("{id}")]
public async Task<IActionResult> UpdateUser(int id, UpdateUserDto
dto)
{
    var user = await _context.Users.FindAsync(id);
    if (user == null)
        return NotFound();

    user.FirstName = dto.FirstName;
    user.LastName = dto.LastName;
    user.RoleId = dto.RoleId;
    user.IsActive = dto.IsActive;
    user.UpdatedAt = DateTime.UtcNow;

    await _context.SaveChangesAsync();

    return NoContent();
}

// DELETE: api/users/5
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteUser(int id)
```

```
{
    var user = await _context.Users.FindAsync(id);
    if (user == null)
        return NotFound();

    // Soft delete
    user.IsActive = false;
    user.UpdatedAt = DateTime.UtcNow;

    await _context.SaveChangesAsync();

    return NoContent();
}

}
```

DTOs (Data Transfer Objects)

```
public record UserDto
{
public int Id { get; init; }
public string Email { get; init; }
public string FirstName { get; init; }
public string LastName { get; init; }
public string RoleName { get; init; }
public bool IsActive { get; init; }
public DateTime CreatedAt { get; init; }
}
public record CreateUserDto
{
[Required]
[EmailAddress]
public string Email { get; init; }
[Required]
[MinLength(6)]
public string Password { get; init; }

[Required]
public string FirstName { get; init; }

[Required]
public string LastName { get; init; }

[Required]
public int RoleId { get; init; }

}
public record UpdateUserDto
{
[Required]
public string FirstName { get; init; }
[Required]
public string LastName { get; init; }

[Required]
public int RoleId { get; init; }
```

```
public bool IsActive { get; init; } = true;

}
public record PagedResult<T>
{
public List<T> Items { get; init; }
public int TotalCount { get; init; }
public int Page { get; init; }
public int PageSize { get; init; }
public int TotalPages => (int)Math.Ceiling((double)TotalCount /
PageSize);
}
```

API Routes en Next.js

Proxy desde Next.js (app/api/users/route.ts):

```
import { NextRequest, NextResponse } from 'next/server';
import { getServerSession } from 'next-auth';

const API_BASE = process.env.BACKEND_API_URL ||
'http://localhost:5000';

export async function GET(request: NextRequest) {
const session = await getServerSession();
if (!session) {
return NextResponse.json({ error: 'Unauthorized' }, { status: 401
});
}

const { searchParams } = new URL(request.url);
const page = searchParams.get('page') || '1';
const pageSize = searchParams.get('pageSize') || '10';
const search = searchParams.get('search') || '';

const response = await fetch(
${API_BASE}/api/users?page=${page}&pageSize=${pageSize}&search=${sea
rch},
{
headers: {
Authorization: Bearer ${session.accessToken},
},
}
);

const data = await response.json();
return NextResponse.json(data);
}

export async function POST(request: NextRequest) {
const session = await getServerSession();
if (!session || session.user.role !== 'admin') {
```

```
return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}

const body = await request.json();

const response = await fetch(${API_BASE}/api/users, {
method: 'POST',
headers: {
'Content-Type': 'application/json',
Authorization: Bearer ${session.accessToken},
},
body: JSON.stringify(body),
});

const data = await response.json();
return NextResponse.json(data, { status: response.status });
}
```

Ejercicio Práctico

Actividad: Crear API REST completa

1. Implementar controlador con endpoints CRUD
2. Crear DTOs con validaciones
3. Probar endpoints con Postman/Thunder Client
4. Implementar paginación y búsqueda
5. Crear API routes en Next.js como proxy

4.3 Interfaz de Administración de Usuarios (2.5 horas)

Tabla Interactiva con TanStack Table

Instalación:

npm install @tanstack/react-table

Componente UserTable (components/users/UserTable.tsx):

```
'use client';
import {
useReactTable,
getCoreRowModel,
getFilteredRowModel,
getPaginationRowModel,
flexRender,
ColumnDef,
} from '@tanstack/react-table';
import { useState } from 'react';
```

```
interface User {
id: number;
email: string;
firstName: string;
lastName: string;
roleName: string;
isActive: boolean;
}
const columns: ColumnDef<User>[] = [
{
accessorKey: 'id',
header: 'ID',
},
{
accessorKey: 'email',
header: 'Email',
},
{
accessorFn: (row) => ${row.firstName} ${row.lastName},
id: 'fullName',
header: 'Nombre Completo',
},
{
accessorKey: 'roleName',
header: 'Rol',
},
{
accessorKey: 'isActive',
header: 'Estado',
cell: ({ getValue }) => (
<span
className={px-2 py-1 rounded text-xs ${ getValue() ? 'bg-green-100
text-green-800' : 'bg-red-100 text-red-800' }}
>
{getValue() ? 'Activo' : 'Inactivo'}
</span>
),
},
{
id: 'actions',
header: 'Acciones',
cell: ({ row }) => (


<button
onClick={() => onEdit(row.original)}
className="text-blue-600 hover:underline"
>
Editar
</button>
<button
onClick={() => onDelete(row.original)}
className="text-red-600 hover:underline"
>
```

```
Eliminar
</button>

),
},
];
export default function UserTable({ users }: { users: User[] }) {
const [globalFilter, setGlobalFilter] = useState('');
const table = useReactTable({
data: users,
columns,
state: {
globalFilter,
},
onGlobalFilterChange: setGlobalFilter,
getCoreRowModel: getCoreRowModel(),
getFilteredRowModel: getFilteredRowModel(),
getPaginationRowModel: getPaginationRowModel(),
});
return (
<div>
{/* Buscador */}


<input
value={globalFilter ?? ''}
onChange={(e) => setGlobalFilter(e.target.value)}
placeholder="Buscar usuarios..."
className="w-full max-w-sm px-3 py-2 border rounded"
/>
  {/* Tabla */}
  <div className="overflow-x-auto">
    <table className="min-w-full divide-y divide-gray-200">
      <thead className="bg-gray-50">
        {table.getHeaderGroups().map((headerGroup) => (
          <tr key={headerGroup.id}>
            {headerGroup.headers.map((header) => (
              <th
                key={header.id}
                className="px-6 py-3 text-left text-xs font-medium
text-gray-500 uppercase"
              >
                {flexRender(
                  header.column.columnDef.header,
                  header.getContext()
                )}
              </th>
            ))}
          </tr>
        ))}
      </thead>
      <tbody className="bg-white divide-y divide-gray-200">
        {table.getRowModel().rows.map((row) => (
          <tr key={row.id}>
```

```
            {row.getVisibleCells().map((cell) => (
              <td key={cell.id} className="px-6 py-4 whitespace-
nowrap">
                {flexRender(cell.column.columnDef.cell,
cell.getContext())}
              </td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  </div>

  {/* Paginación */}
  <div className="flex items-center justify-between mt-4">
    <div className="flex gap-2">
      <button
        onClick={() => table.previousPage()}
        disabled={!table.getCanPreviousPage()}
        className="px-3 py-1 border rounded disabled:opacity-50"
      >
        Anterior
      </button>
      <button
        onClick={() => table.nextPage()}
        disabled={!table.getCanNextPage()}
        className="px-3 py-1 border rounded disabled:opacity-50"
      >
        Siguiente
      </button>
    </div>
    <span className="text-sm text-gray-700">
      Página {table.getState().pagination.pageIndex + 1} de{' '}
      {table.getPageCount()}
    </span>
  </div>
</div>

);
}
```

Formulario de Creación/Edición

```
'use client';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import toast from 'react-hot-toast';
const userSchema = z.object({
email: z.string().email('Email inválido'),
firstName: z.string().min(2, 'Nombre muy corto'),
lastName: z.string().min(2, 'Apellido muy corto'),
```

```
password: z.string().min(6, 'Mínimo 6 caracteres').optional(),
roleId: z.number().min(1, 'Seleccione un rol'),
});
type UserFormData = z.infer<typeof userSchema>;
export default function UserForm({
user,
onClose,
}: {
user?: UserFormData;
onClose: () => void;
}) {
const {
register,
handleSubmit,
formState: { errors, isSubmitting },
} = useForm<UserFormData>({
resolver: zodResolver(userSchema),
defaultValues: user,
});
const onSubmit = async (data: UserFormData) => {
try {
const method = user ? 'PUT' : 'POST';
const url = user ? /api/users/${user.id} : '/api/users';
  const response = await fetch(url, {
    method,
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });

  if (!response.ok) throw new Error('Error al guardar usuario');

  toast.success(user ? 'Usuario actualizado' : 'Usuario creado');
  onClose();
} catch (error) {
  toast.error('Error al guardar');
}

};
return (
<form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
Email
<input
{...register('email')}
type="email"
className="w-full px-3 py-2 border rounded"
/>
{errors.email && (

{errors.email.message}

)}

  <div>
    <label className="block text-sm font-medium mb-1">Nombre</label>
```

```
    <input
      {...register('firstName')}
      className="w-full px-3 py-2 border rounded"
    />
    {errors.firstName && (
      <p className="text-red-500 text-
sm">{errors.firstName.message}</p>
    )}
  </div>

  <div>
    <label className="block text-sm font-medium mb-
1">Apellido</label>
    <input
      {...register('lastName')}
      className="w-full px-3 py-2 border rounded"
    />
    {errors.lastName && (
      <p className="text-red-500 text-
sm">{errors.lastName.message}</p>
    )}
  </div>

  {!user && (
    <div>
      <label className="block text-sm font-medium mb-
1">Contraseña</label>
      <input
        {...register('password')}
        type="password"
        className="w-full px-3 py-2 border rounded"
      />
      {errors.password && (
        <p className="text-red-500 text-
sm">{errors.password.message}</p>
      )}
    </div>
  )}

  <div>
    <label className="block text-sm font-medium mb-1">Rol</label>
    <select
      {...register('roleId', { valueAsNumber: true })}
      className="w-full px-3 py-2 border rounded"
    >
      <option value="">Seleccione...</option>
      <option value={1}>Admin</option>
      <option value={2}>Usuario</option>
    </select>
    {errors.roleId && (
      <p className="text-red-500 text-
sm">{errors.roleId.message}</p>
    )}
  </div>
```

```
  <div className="flex gap-2 justify-end">
    <button
      type="button"
      onClick={onClose}
      className="px-4 py-2 border rounded"
    >
      Cancelar
    </button>
    <button
      type="submit"
      disabled={isSubmitting}
      className="px-4 py-2 bg-blue-600 text-white rounded
disabled:opacity-50"
    >
      {isSubmitting ? 'Guardando...' : 'Guardar'}
    </button>
  </div>
</form>

);
}
```

Modal de Confirmación

```
'use client';
export default function ConfirmDialog({
title,
message,
onConfirm,
onCancel,
}: {
title: string;
message: string;
onConfirm: () => void;
onCancel: () => void;
}) {
return (

{title}
{message}

Cancelar

Confirmar


);
}
```

Ejercicio Práctico

Actividad: Construir UI completa

1. Instalar TanStack Table
2. Crear componente UserTable con columnas
3. Implementar buscador y paginación
4. Crear formulario con validación Zod
5. Agregar modal de confirmación para eliminar
6. Conectar todo con API routes
7. Probar flujo completo CRUD

4.4 Validación y Seguridad (0.5 horas)

Validación en Frontend

Schemas Zod compartidos (lib/schemas.ts):

```
import { z } from 'zod';
export const emailSchema = z.string().email('Email inválido');
export const passwordSchema = z
.string()
.min(8, 'Mínimo 8 caracteres')
.regex(/[A-Z]/, 'Debe contener mayúscula')
.regex(/[a-z]/, 'Debe contener minúscula')
.regex(/[0-9]/, 'Debe contener número')
.regex(/[^A-Za-z0-9]/, 'Debe contener carácter especial');
export const userSchema = z.object({
email: emailSchema,
firstName: z.string().min(2).max(50),
lastName: z.string().min(2).max(50),
password: passwordSchema.optional(),
roleId: z.number().positive(),
});
```

Sanitización de Inputs

```
import DOMPurify from 'isomorphic-dompurify';
export function sanitizeInput(input: string): string {
return DOMPurify.sanitize(input, {
ALLOWED_TAGS: [], // No HTML tags allowed
ALLOWED_ATTR: [],
});
}
// Uso
const cleanEmail = sanitizeInput(userInput);
```

Prevención de XSS

- ✅ Usar textContent en lugar de innerHTML
- ✅ Validar y sanitizar todos los inputs

- ✅ Escapar caracteres especiales en HTML
- ✅ Usar Content Security Policy headers
- ❌ NUNCA usar dangerouslySetInnerHTML sin sanitizar

Prevención de SQL Injection

Entity Framework protege automáticamente usando queries parametrizadas:

```
// ✅ SEGURO - EF Core usa parámetros
var users = await _context.Users
.Where(u => u.Email == userEmail) // Parametrizado
.ToListAsync();
// ❌ INSEGURO - SQL raw sin parametrizar
var users = await _context.Users
.FromSqlRaw($"SELECT * FROM Users WHERE Email = '{userEmail}'")
.ToListAsync();
// ✅ SEGURO - SQL raw con parámetros
var users = await _context.Users
.FromSqlRaw("SELECT * FROM Users WHERE Email = {0}", userEmail)
.ToListAsync();
```

Rate Limiting Básico

Middleware Next.js (middleware.ts):

```
import { NextRequest, NextResponse } from 'next/server';
import { Ratelimit } from '@upstash/ratelimit';
import { Redis } from '@upstash/redis';
const ratelimit = new Ratelimit({
redis: Redis.fromEnv(),
limiter: Ratelimit.slidingWindow(10, '10 s'), // 10 requests por 10
segundos
});
export async function middleware(request: NextRequest) {
if (request.nextUrl.pathname.startsWith('/api/')) {
const ip = request.ip ?? '127.0.0.1';
const { success } = await ratelimit.limit(ip);
if (!success) {
  return NextResponse.json(
    { error: 'Too many requests' },
    { status: 429 }
  );
}

}
return NextResponse.next();
}
```

Ejercicio Práctico

Actividad: Fortalecer seguridad

1. Crear schemas Zod robustos
2. Agregar sanitización en formularios
3. Implementar validación de contraseña fuerte
4. Verificar que EF Core usa queries parametrizadas
5. Probar inyección XSS (debe fallar)

5. SESIÓN 4: UI/UX MODERNA Y DESPLIEGUE

Duración: 5 horas
Día: Martes (siguiente semana), 8:00 AM - 1:00 PM

5.1 Interfaces Modernas con Tailwind y shadcn/ui (2 horas)

Introducción a Tailwind CSS

Tailwind es un framework CSS utility-first[19] que permite diseñar interfaces directamente en HTML sin escribir CSS custom.

Ventajas:

- Desarrollo rápido (no pensar nombres de clases)
- Bundle CSS optimizado (solo clases usadas)
- Diseño consistente con sistema de design tokens
- Responsive design fácil
- Dark mode built-in

shadcn/ui: Componentes Accesibles

shadcn/ui NO es una librería de componentes tradicional[20][21]. Es una colección de componentes que copias a tu proyecto.

Diferencias con otras librerías:

Aspecto	shadcn/ui	Material-UI	Chakra UI
Instalación	Copias código	npm install	npm install
Propiedad	Tú eres dueño	Dependencia	Dependencia
Personalización	Total	Limitada	Media
Bundle size	Mínimo	Grande	Medio

Table 6: Comparación de librerías UI

Instalación:

npx shadcn-ui@latest init

Configuración interactiva:

- Style: Default
- Base color: Slate
- CSS variables: Yes

Agregar componentes:

npx shadcn-ui@latest add button
npx shadcn-ui@latest add input
npx shadcn-ui@latest add dialog
npx shadcn-ui@latest add table
npx shadcn-ui@latest add form
npx shadcn-ui@latest add select

Sistema de Diseño

Tokens de color (tailwind.config.ts):

```
export default {
theme: {
extend: {
colors: {
// Brand colors
primary: {
50: '#f0f9ff',
100: '#e0f2fe',
500: '#3b82f6',
600: '#2563eb',
900: '#1e3a8a',
},
// Neutrals
gray: {
50: '#f9fafb',
100: '#f3f4f6',
500: '#6b7280',
900: '#111827',
},
},
fontFamily: {
sans: ['Inter', 'system-ui', 'sans-serif'],
mono: ['Fira Code', 'monospace'],
},
spacing: {
18: '4.5rem',
```

```
88: '22rem',
},
},
},
};
```

Componentes reutilizables con shadcn/ui:

```
// components/ui/button.tsx (generado por shadcn)
import * as React from 'react';
import { cva, type VariantProps } from 'class-variance-authority';
import { cn } from '@/lib/utils';

const buttonVariants = cva(
'inline-flex items-center justify-center rounded-md text-sm font-
medium transition-colors focus-visible:outline-none
disabled:pointer-events-none disabled:opacity-50',
{
variants: {
variant: {
default: 'bg-primary text-white hover:bg-primary/90',
destructive: 'bg-red-500 text-white hover:bg-red-600',
outline: 'border border-gray-300 hover:bg-gray-100',
ghost: 'hover:bg-gray-100',
},
size: {
default: 'h-10 px-4 py-2',
sm: 'h-9 px-3',
lg: 'h-11 px-8',
icon: 'h-10 w-10',
},
},
defaultVariants: {
variant: 'default',
size: 'default',
},
}
);

export interface ButtonProps
extends React.ButtonHTMLAttributes<HTMLButtonElement>,
VariantProps<typeof buttonVariants> {}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
({ className, variant, size, ...props }, ref) => {
return (
<button
className={cn(buttonVariants({ variant, size, className }))}
ref={ref}
{...props}
/>
);
}
);
```

```
Button.displayName = 'Button';

export { Button, buttonVariants };
```

Uso:

import { Button } from '@/components/ui/button';

Default Delete Small Outline

Dark Mode con next-themes

Instalación:

npm install next-themes

Provider (app/providers.tsx):

```
'use client';

import { ThemeProvider } from 'next-themes';

export function Providers({ children }: { children: React.ReactNode
}) {
return (

{children}

);
}
```

Layout (app/layout.tsx):

```
import { Providers } from './providers';

export default function RootLayout({ children }) {
return (
{children}
);
}
```

Toggle button:

```
'use client';

import { useTheme } from 'next-themes';
import { useEffect, useState } from 'react';

export function ThemeToggle() {
const [mounted, setMounted] = useState(false);
const { theme, setTheme } = useTheme();

useEffect(() => setMounted(true), []);

if (!mounted) return null;
```

```
return (
<button
onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}
className="p-2 rounded hover:bg-gray-100 dark:hover:bg-gray-800"
>
{theme === 'dark' ? '🌞' : '🌙'}
</button>
);
}
```

CSS para dark mode (globals.css):

```
@layer base {
:root {
--background: 0 0% 100%;
--foreground: 222.2 84% 4.9%;
}

.dark {
--background: 222.2 84% 4.9%;
--foreground: 210 40% 98%;
}
}
```

Diseño Responsive

Mobile-first approach con Tailwind:

{/* Cards */}

Guardar

Breakpoints de Tailwind:

- sm: 640px
- md: 768px
- lg: 1024px
- xl: 1280px
- 2xl: 1536px

Accesibilidad (a11y)

ARIA labels:

```
<button
aria-label="Cerrar modal"
onClick={onClose}

</button>
```

```
{errors.email && (
{errors.email.message}
)}
```

Keyboard navigation:

```
{ if (e.key === 'Enter' || e.key === ' ') { handleClick(); } }} >
Clickeable div
```

Focus indicators:

```
.focus-visible:focus {
@apply outline-2 outline-offset-2 outline-blue-600;
}
```

Recursos de Aprendizaje

Documentación:

- Tailwind CSS: https://tailwindcss.com/docs
- shadcn/ui: https://ui.shadcn.com/docs[22]
- next-themes: https://github.com/pacocoursey/next-themes

Artículos:

- "Construyendo UIs Accesibles con Shadcn UI": https://4geeks.com/es/lesson/construyendo-componentes-con-shadcn[23]

Videos:

- "DEBERÍAS USAR SHADCN YA": https://www.youtube.com/watch?v=RYCEHMF6nyw[24]
- "Shadcn/ui: Componentes accesibles": https://www.youtube.com/watch?v=K1SmRHUpsLM[25]
- "Curso Shadcn/ui - DevTalles": https://cursos.devtalles.com/courses/shadcn-ui[26]

Ejercicio Práctico

Actividad: Modernizar UI completa

1. Instalar shadcn/ui y agregar componentes básicos
2. Migrar botones y formularios a shadcn/ui
3. Configurar sistema de colores personalizado
4. Implementar dark mode con next-themes
5. Hacer todas las páginas responsive
6. Agregar ARIA labels donde falten
7. Probar navegación por teclado (Tab, Enter)

5.2 Optimización de Rendimiento (1 hora)

Code Splitting y Lazy Loading

Componentes dinámicos:

```
import dynamic from 'next/dynamic';
// Lazy load con loading state
const HeavyChart = dynamic(() => import('@/components/HeavyChart'),
{
loading: () =>
Cargando gráfico...
,
ssr: false, // Deshabilitar SSR si componente usa window
});
export default function DashboardPage() {
return (
Dashboard
);
}
```

Lazy loading condicional:

```
'use client';
import { useState } from 'react';
import dynamic from 'next/dynamic';
const AdminPanel = dynamic(() => import('@/components/AdminPanel'));
export default function Dashboard() {
const [showAdmin, setShowAdmin] = useState(false);
return (


<button onClick={() => setShowAdmin(true)}>
Mostrar Panel Admin
</button>
{showAdmin && }
);
}
```

Optimización de Imágenes

Componente next/image:

```
import Image from 'next/image';
<Image
src="/hero.jpg"
alt="Hero image"
width={1200}
height={600}
priority // Cargar inmediatamente (above the fold)
placeholder="blur" // Efecto blur mientras carga
blurDataURL="..." // Base64 del placeholder
/>
{/* Imagen externa */}
<Image
src="https://api.infiniteherbs.com/images/product.jpg"
alt="Product"
```

```
width={400}
height={400}
// Requiere configurar domains en next.config.js
/>
```

next.config.js:

```
module.exports = {
images: {
domains: ['api.infiniteherbs.com'],
formats: ['image/avif', 'image/webp'], // Formatos modernos
deviceSizes: [640, 750, 828, 1080, 1200], // Tamaños responsive
},
};
```

Caching y Revalidación

Estrategias de cache:

```
// 1. Forzar SSG (Static) - cache infinito
export const revalidate = false;
async function getData() {
const res = await fetch('https://api.example.com/data');
return res.json();
}
// 2. ISR (Incremental Static Regeneration) - revalidar cada X
segundos
export const revalidate = 3600; // 1 hora
async function getData() {
const res = await fetch('https://api.example.com/data', {
next: { revalidate: 3600 },
});
return res.json();
}
// 3. SSR dinámico - sin cache
async function getData() {
const res = await fetch('https://api.example.com/data', {
cache: 'no-store',
});
return res.json();
}
// 4. Cache con tags (para invalidar programáticamente)
async function getData() {
const res = await fetch('https://api.example.com/data', {
next: { tags: ['users'] },
});
return res.json();
}
// Invalidar cache
import { revalidateTag } from 'next/cache';
revalidateTag('users');
```

Web Vitals y Lighthouse

Métricas clave:

Métrica	Descripción	Objetivo
LCP (Largest Contentful Paint)	Tiempo hasta contenido principal visible	<2.5s
FID (First Input Delay)	Tiempo hasta primera interacción	<100ms
CLS (Cumulative Layout Shift)	Estabilidad visual (sin saltos)	<0.1
TTFB (Time To First Byte)	Tiempo respuesta servidor	<600ms

Table 7: Core Web Vitals

Medir en desarrollo:

```
// app/layout.tsx
import { Analytics } from '@vercel/analytics/react';

export default function RootLayout({ children }) {
return (

{children}

);
}
```

Custom reporting:

```
// app/web-vitals.ts
import { useReportWebVitals } from 'next/web-vitals';

export function WebVitals() {
useReportWebVitals((metric) => {
console.log(metric);
// Enviar a analytics
fetch('/api/analytics', {
method: 'POST',
body: JSON.stringify(metric),
});
});
}
```

Recursos de Aprendizaje

Documentación:

- Image Optimization: https://nextjs.org/docs/app/building-your-application/optimizing/images
- Code Splitting: https://nextjs.org/docs/app/building-your-application/optimizing/lazy-loading
- Caching: https://nextjs.org/docs/app/building-your-application/caching
- Web Vitals: https://web.dev/vitals/

Ejercicio Práctico

Actividad: Optimizar aplicación

1. Migrar todas las imágenes a next/image
2. Lazy load componente pesado (tabla usuarios)
3. Configurar revalidación ISR en página principal
4. Ejecutar Lighthouse y analizar score
5. Mejorar LCP y CLS basado en sugerencias

5.3 Despliegue en IIS (1.5 horas)

Build de Producción

Preparación:

- **Instalar dependencias**
  npm install
- **Build optimizado**
  npm run build
- **Probar localmente**
  npm start

Resultado del build:

```
.next/
├── server/ # Código servidor
├── static/ # Assets estáticos
└── standalone/ # Build autónomo (opcional)
```

Configuración de IIS para Next.js

Requisitos en Windows Server:

1. Node.js 20.x instalado
2. IIS instalado y habilitado
3. iisnode instalado: https://github.com/Azure/iisnode
4. URL Rewrite módulo: https://www.iis.net/downloads/microsoft/url-rewrite

Instalar iisnode:

Descargar desde GitHub releases

Ejecutar instalador .msi

iisnode-full-v0.2.26-x64.msi

Configuración de IIS:

1. Abrir IIS Manager
2. Crear nuevo sitio web:
   - Nombre: InfiniteHerbsApp
   - Ruta física: C:\inetpub\wwwroot\infinite-herbs
   - Puerto: 80 (o el deseado)
3. Configurar Application Pool:
   - .NET CLR Version: No Managed Code
   - Pipeline Mode: Integrated

web.config para Next.js

Crear web.config en raíz del proyecto:

Archivo server.js

Crear server.js en raíz:

```
const { createServer } = require('http');
const next = require('next');

const dev = process.env.NODE_ENV !== 'production';
const app = next({ dev });
const handle = app.getRequestHandler();

const port = process.env.PORT || 3000;

app.prepare().then(() => {
createServer((req, res) => {
handle(req, res);
}).listen(port, (err) => {
if (err) throw err;
console.log(> Ready on http://localhost:${port});
});
});
```

Variables de Entorno en Producción

Opción 1: Archivo .env.production en servidor

C:\inetpub\wwwroot\infinite-herbs.env.production

```
DATABASE_URL="Server=prod-db;Database=InfiniteHerbs;..."
API_SECRET_KEY="production-secret-key"
NEXTAUTH_URL="https://app.infiniteherbs.com"
NEXTAUTH_SECRET="production-nextauth-secret"
```

Opción 2: Variables de entorno de Windows

PowerShell (requiere permisos admin)

```
[System.Environment]::SetEnvironmentVariable('DATABASE_URL', 'Server=...', 'Machine')
[System.Environment]::SetEnvironmentVariable('API_SECRET_KEY', 'secret', 'Machine')
```

Reiniciar IIS

```
iisreset
```

HTTPS y Certificados SSL

Configuración SSL en IIS:

1. Obtener certificado SSL (Let's Encrypt, Azure, etc.)
2. Importar certificado a Windows Certificate Store
3. En IIS Manager:
   - Seleccionar sitio
   - Bindings → Add
   - Type: https
   - Port: 443
   - SSL Certificate: seleccionar certificado

Redirección HTTP → HTTPS (web.config):

Recursos de Aprendizaje

Documentación:

- Next.js Deployment: https://nextjs.org/docs/app/building-your-application/deploying
- iisnode GitHub: https://github.com/Azure/iisnode

Videos:

- "Deploy Next.js to Windows Server IIS on AWS": https://www.youtube.com/watch?v=rbZf5kzrK_o[27]
- "Cara mudah deploy Next JS ke IIS": https://www.youtube.com/watch?v=XxR3FBRS3C0[28]

GitHub:

- "nextjs-windows-server-setup": https://github.com/errunlee/nextjs-windows-server-setup[29]

Ejercicio Práctico

Actividad: Simular despliegue

1. Ejecutar npm run build localmente
2. Crear web.config y server.js
3. Configurar variables de entorno de producción
4. Probar con npm start (simulando servidor)
5. Documentar pasos para despliegue real en IIS
6. (Si hay acceso a servidor) Realizar despliegue real

5.4 Mejores Prácticas y Troubleshooting (0.5 horas)

Testing Básico

Instalación de Jest + React Testing Library:

npm install -D jest @testing-library/react @testing-library/jest-dom jest-environment-jsdom

Configuración (jest.config.js):

```
const nextJest = require('next/jest');

const createJestConfig = nextJest({
dir: './',
});

const customJestConfig = {
setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
testEnvironment: 'jest-environment-jsdom',
moduleNameMapper: {
'^@/(.*)$': '<rootDir>/$1',
},
};

module.exports = createJestConfig(customJestConfig);
```

jest.setup.js:

import '@testing-library/jest-dom';

Ejemplo de test (components/Button.test.tsx):

```
import { render, screen, fireEvent } from '@testing-library/react';
import { Button } from './Button';

describe('Button', () => {
it('renders with text', () => {
render(Click me);
expect(screen.getByText('Click me')).toBeInTheDocument();
});

it('calls onClick handler', () => {
const handleClick = jest.fn();
render(Click);
fireEvent.click(screen.getByText('Click'));
expect(handleClick).toHaveBeenCalledTimes(1);
});

it('applies variant styles', () => {
render(Delete);
const button = screen.getByText('Delete');
expect(button).toHaveClass('bg-red-500');
```

```
});
});
```

Git Workflow

Branching strategy:

```
main (producción)
├── develop (desarrollo)
│   ├── feature/user-crud
│   ├── feature/auth-system
│   └── bugfix/login-validation
└── hotfix/critical-bug
```

Comandos básicos:

Crear feature branch

```
git checkout -b feature/user-management
```

Commits descriptivos

```
git commit -m "feat: add user CRUD endpoints"
git commit -m "fix: resolve login validation issue"
git commit -m "docs: update API documentation"
```

Pull request y code review

```
git push origin feature/user-management
```

Crear PR en GitHub/GitLab

Merge a develop después de aprobación

```
git checkout develop
git merge feature/user-management
```

Logging y Monitoreo

Winston logger (lib/logger.ts):

```
import winston from 'winston';

export const logger = winston.createLogger({
level: process.env.LOG_LEVEL || 'info',
format: winston.format.combine(
winston.format.timestamp(),
winston.format.errors({ stack: true }),
winston.format.json()
),
transports: [
new winston.transports.File({ filename: 'logs/error.log', level:
'error' }),
```

```
new winston.transports.File({ filename: 'logs/combined.log' }),
],
});

if (process.env.NODE_ENV !== 'production') {
logger.add(
new winston.transports.Console({
format: winston.format.simple(),
})
);
}
```

Uso:

```
import { logger } from '@/lib/logger';

// En API routes
export async function POST(request: NextRequest) {
try {
logger.info('Creating new user', { email: data.email });
// ...
} catch (error) {
logger.error('Failed to create user', { error, email: data.email });
return NextResponse.json({ error: 'Internal error' }, { status: 500
});
}
}
```

Problemas Comunes y Soluciones

Problema	Solución
Error "Module not found"	Verificar tsconfig.json paths, reinstalar node_modules
Hydration mismatch	Asegurar mismo HTML en servidor y cliente, usar useEffect para código cliente
API route 404	Verificar estructura app/api/, reiniciar servidor dev
CORS errors	Configurar CORS en backend, verificar credenciales
Build falla	Limpiar .next/, verificar errores TypeScript, npm run build
Sesión no persiste	Configurar NEXTAUTH_SECRET, verificar cookies, NEXTAUTH_URL correcto
Imágenes no cargan	Configurar domains en next.config.js
Lentitud en dev	Reducir imports, usar dynamic imports, aumentar RAM

Table 8: Troubleshooting común

Recursos de Aprendizaje

Documentación:

- Testing: https://nextjs.org/docs/app/building-your-application/testing
- Jest: https://jestjs.io/docs/getting-started
- React Testing Library: https://testing-library.com/react

Ejercicio Práctico

Actividad: Setup completo de calidad

1. Instalar Jest y React Testing Library
2. Escribir 3 tests para componente Button
3. Configurar Git con .gitignore apropiado
4. Crear feature branch y hacer commit
5. Instalar Winston y agregar logging en API route
6. Probar y resolver un error común de la tabla

6. RECURSOS ADICIONALES Y REFERENCIAS

6.1 Documentación Oficial

Next.js:

- Documentación principal: https://nextjs.org/docs
- Learn Next.js (tutorial interactivo): https://nextjs.org/learn
- Examples (repositorio oficial): https://github.com/vercel/next.js/tree/canary/examples

React:

- React Docs: https://react.dev
- React Server Components: https://react.dev/reference/rsc/server-components

TypeScript:

- TypeScript Handbook: https://www.typescriptlang.org/docs/
- TypeScript con React: https://react-typescript-cheatsheet.netlify.app

Entity Framework Core:

- EF Core Docs: https://learn.microsoft.com/es-es/ef/core/
- Migraciones: https://learn.microsoft.com/es-es/ef/core/managing-schemas/migrations/

6.2 Librerías Utilizadas

Librería	Versión	Propósito
Next.js	16+	Framework React full-stack
React	19+	Librería UI
TypeScript	5+	Type safety
Tailwind CSS	4+	Utility-first CSS
shadcn/ui	Latest	Componentes UI accesibles
React Hook Form	7+	Manejo de formularios
Zod	3+	Validación de schemas
TanStack Table	8+	Tablas interactivas
react-hot-toast	2+	Notificaciones

Table 9: Stack tecnológico principal

6.3 Cursos en Español Recomendados

Plataformas:

- DevTalles (Fernando Herrera): https://cursos.devtalles.com
  - Next.js Framework React (actualizado constantemente)
  - React de Cero a Experto
  - TypeScript Guía Completa
  - Shadcn/ui Componentes Accesibles
- Udemy en Español:
  - Buscar "Next.js" + "español"
  - Verificar fecha de actualización (2024-2026)

YouTube:

- Fernando Herrera (DevTalles): https://www.youtube.com/@DevTalles
- midudev: https://www.youtube.com/@midudev
- Gentleman Programming: https://www.youtube.com/@GentlemanProgramming

6.4 Comunidades y Soporte

Discord:

- Next.js Discord: https://discord.gg/nextjs
- DevTalles Discord: https://discord.gg/pBjEVYTC7t

Foros:

- Stack Overflow (tag: next.js): https://stackoverflow.com/questions/tagged/next.js
- GitHub Discussions Next.js: https://github.com/vercel/next.js/discussions

Reddit:

- r/nextjs: https://reddit.com/r/nextjs
- r/reactjs: https://reddit.com/r/reactjs

6.5 Herramientas de Desarrollo

Code Editors:

- Visual Studio Code: https://code.visualstudio.com
- WebStorm: https://www.jetbrains.com/webstorm/

API Testing:

- Postman: https://www.postman.com
- Thunder Client (VS Code extension)
- Insomnia: https://insomnia.rest

Database Management:

- SQL Server Management Studio (SSMS)
- Azure Data Studio: https://azure.microsoft.com/es-es/products/data-studio
- DBeaver: https://dbeaver.io

DevTools:

- React Developer Tools (browser extension)
- Redux DevTools (si usas Redux)
- Lighthouse (Chrome DevTools)

7. GLOSARIO DE TÉRMINOS

API (Application Programming Interface): Interfaz que permite comunicación entre diferentes sistemas de software.

App Router: Nueva arquitectura de routing de Next.js 13+ basada en carpeta app/.

ARIA (Accessible Rich Internet Applications): Estándares para hacer aplicaciones web accesibles.

Authentication: Proceso de verificar identidad de un usuario (quién eres).

Authorization: Proceso de verificar permisos de un usuario (qué puedes hacer).

Client Component: Componente React que se renderiza y ejecuta en el navegador, permite interactividad.

CORS (Cross-Origin Resource Sharing): Mecanismo de seguridad para permitir/bloquear requests entre dominios diferentes.

CRUD: Create, Read, Update, Delete - operaciones básicas de base de datos.

CSR (Client-Side Rendering): Renderizado que ocurre en el navegador del usuario.

Dark Mode: Esquema de colores oscuro para interfaces, reduce fatiga visual.

DTO (Data Transfer Object): Objeto que transporta datos entre capas de aplicación.

Entity Framework Core: ORM de Microsoft para .NET que facilita trabajo con bases de datos.

Hydration: Proceso de convertir HTML estático en aplicación React interactiva.

ISR (Incremental Static Regeneration): Estrategia que permite regenerar páginas estáticas sin rebuild completo.

JWT (JSON Web Token): Formato de token para transmitir información de forma segura.

Middleware: Función que intercepta requests antes de llegar a rutas, útil para autenticación.

ORM (Object-Relational Mapping): Técnica de mapear objetos de código a tablas de base de datos.

Pages Router: Sistema de routing legacy de Next.js basado en carpeta pages/.

Props: Propiedades que se pasan de componente padre a hijo en React.

Rate Limiting: Técnica de limitar número de requests por usuario/IP en cierto tiempo.

Responsive Design: Diseño que se adapta a diferentes tamaños de pantalla.

REST API: Arquitectura de API que usa HTTP methods (GET, POST, PUT, DELETE).

Route Handlers: API routes en Next.js App Router (app/api/).

Server Component: Componente React que solo se renderiza en servidor, no envía JavaScript al cliente.

SSG (Static Site Generation): Pre-renderizado de páginas en build time.

SSR (Server-Side Rendering): Renderizado de páginas en cada request del servidor.

State: Datos internos de un componente que pueden cambiar y causar re-render.

Tailwind CSS: Framework CSS utility-first para diseño rápido.

TypeScript: Superset de JavaScript con tipos estáticos.

Zod: Librería de validación y schemas para TypeScript.

Referencias

[1] Vercel. (2026). *Next.js Documentation*. https://nextjs.org/docs

[2] Vercel. (2026). *Next.js Docs - Getting Started*. https://nextjs.org/docs/app/getting-started

[3] Next.js Español. (2024). *Introducción - Next.js Español*. https://es.nextjs.im/docs

[4] Herrera, F. (2024). *Next.js 15 Tutorial para Principiantes 2025*. YouTube. https://www.youtube.com/watch?v=c_DPfd85qTM

[5] Herrera, F. (2023). *Tutorial oficial de NextJS - Capítulo 01*. YouTube. https://www.youtube.com/watch?v=O6FpxIHOAfI

[6] React Team. (2026). *Server Components*. React Documentation. https://react.dev/reference/rsc/server-components

[7] Inhaq. (2026). *React Server Components: A Practical Guide for 2026*. https://inhaq.com/blog/react-server-components-practical-guide-2026

[8] React Team. (2024). *Server Components - React*. https://react.dev/reference/rsc/server-components

[9] Inhaq. (2026). *React Server Components: A Practical Guide for 2026*. https://inhaq.com/blog/react-server-components-practical-guide-2026

[10] Nexgen Systems. (2025). *React Server Components con Next.js 16*. https://nexgen.mx/blog/react-server-components-nextjs

[11] Microsoft. (2025). *Entity Framework Core Documentation*. https://learn.microsoft.com/es-es/ef/core/

[12] Microsoft. (2025). *Tutorial: Introducción a EF Core en una aplicación web de ASP.NET*. https://learn.microsoft.com/es-es/aspnet/core/data/ef-mvc/intro

[13] Kudvenkat. (2019). *Using SQL Server with Entity Framework Core*. YouTube. https://www.youtube.com/watch?v=xMktEpPmadI

[14] Code with Robin. (2025). *Entity Framework Core with .NET 10*. YouTube. https://www.youtube.com/watch?v=RXXrp7eBIhs

[15] Watmore, J. (2021). *.NET 5.0 - Connect to SQL Server with Entity Framework Core*. https://jasonwatmore.com/post/2021/10/21/net-5-connect-to-sql-server-with-entity-framework-core

[16] Nexgen Systems. (2025). *Autenticación Segura en Next.js con NextAuth.js y JWT*. https://nexgen.mx/blog/nextjs-authentication-nextauth

[17] Nexgen Systems. (2025). *Autenticación con NextAuth.js*. https://www.nexgen.mx/blog/nextjs-authentication-nextauth

[18] Coding Tube. (2024). *Tutorial Next Auth V5 - Credentials, JWT, Roles*. YouTube. https://www.youtube.com/watch?v=vkV4wApku5s

[19] Tailwind Labs. (2026). *Tailwind CSS Documentation*. https://tailwindcss.com/docs

[20] shadcn. (2026). *shadcn/ui Installation*. https://ui.shadcn.com/docs/installation

[21] 4Geeks. (2025). *Construyendo UIs Accesibles con Shadcn UI en React*. https://4geeks.com/es/lesson/construyendo-componentes-con-shadcn

[22] shadcn. (2026). *shadcn/ui Documentation*. https://ui.shadcn.com/docs

[23] 4Geeks. (2025). *Construyendo UIs Accesibles con Shadcn UI*. https://4geeks.com/es/lesson/construyendo-componentes-con-shadcn

[24] midudev. (2025). *DEBERÍAS USAR SHADCN YA*. YouTube. https://www.youtube.com/watch?v=RYCEHMF6nyw

[25] Herrera, F. (2024). *Shadcn/ui: Componentes accesibles y personalizables*. YouTube. https://www.youtube.com/watch?v=K1SmRHUpsLM

[26] DevTalles. (2024). *Curso Shadcn/ui - DevTalles*. https://cursos.devtalles.com/courses/shadcn-ui

[27] Code with Robin. (2025). *Deploy Next.js to Windows Server IIS on AWS*. YouTube. https://www.youtube.com/watch?v=rbZf5kzrK_o

[28] Makarim, F. (2024). *Cara mudah deploy Next JS ke IIS windows server*. YouTube. https://www.youtube.com/watch?v=XxR3FBRS3C0

[29] errunlee. (2025). *nextjs-windows-server-setup*. GitHub. https://github.com/errunlee/nextjs-windows-server-setup